

Assessment-2

Q36. Thread-Safe Counter Using Atomic Variables

✧ HEADER:

Name: U. Jayasudha

Roll No:192411228

Course Code & CO Reference: CSA0926 & C02

Course Name: Programming of Java

Assignment set : Direct-Descriptive and Q36

GitHub repository link:<https://github.com/ummadisettijayasudha136-shipit/CSA0926.git>

Submission Date: 11/08/2026

Full Assigned Question:

Q36. Thread-Safe Counter Using Atomic Variables

Write a Java program that solves the same race-condition problem from Q30, but this time using the Atomic Integer class from `java.util.concurrent.atomic` instead of the `synchronized` keyword. Demonstrate that using atomic operations like `increment` And `Get()` also produces a correct final result, and briefly compare this approach to using `synchronized` methods.

Concepts: Atomic Integer, atomic operations, `java.util.concurrent.atomic`.

✧ PROBLEM UNDERSTANDING:

- The program uses several threads to increase one common counter.
- When many threads change the same counter at the same time, some updates can be lost because of a race condition.
- The program uses `AtomicInteger` and `incrementAndGet()` to make every counter update safe and produce the correct final value.

✧ APPROACH / DESIGN NOTES

Class: `AtomicInteger` from `java.util.concurrent.atomic` to perform atomic increment operations

Methods: `incrementAndGet()` – automatically increases the value by 1 and returns the updated value

- ✓ Each thread performs a fixed number of increments on the shared counter.
- ✓ The main thread waits for all threads to finish using `join()`.
- ✓ The final counter value is compared with the expected value ($\text{threads} \times \text{increments per thread}$).

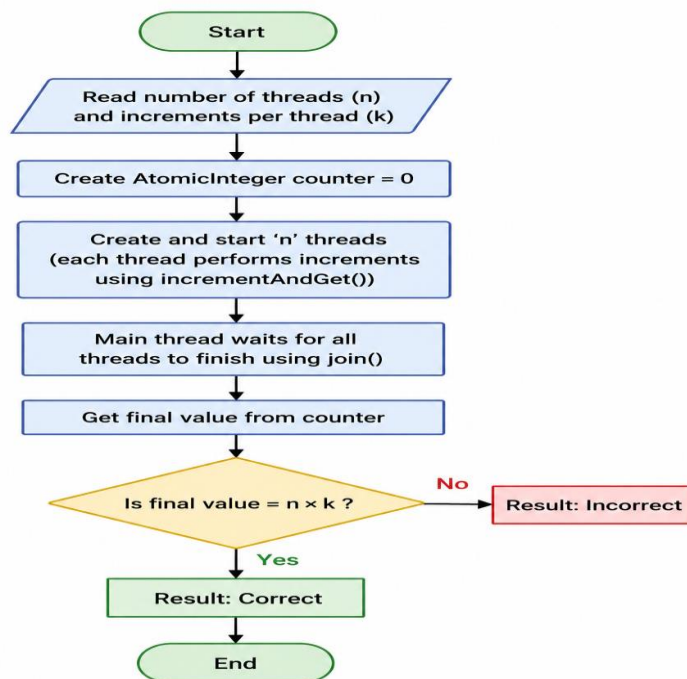
Why AtomicInteger?

AtomicInteger is suitable because it allows the counter to be updated safely without using the synchronized keyword.

Alternative considered:

- I considered using a synchronized method for the counter.
- I chose AtomicInteger because this question specifically focuses on atomic operations and it is simple for updating a shared counter.

Flow chart



✧ SOURCE CODE

```

import java.util.Scanner;
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicCounter {

```

```

static AtomicInteger counter = new AtomicInteger(0);
static class CounterTask implements Runnable {
    private int increments;
    CounterTask(int increments) {
        this.increments = increments;
    }
    public void run() {
        for (int i = 0; i < increments; i++) {
            counter.incrementAndGet();
        }
    }
}

public static void main(String[] args) throws InterruptedException {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of threads: ");
    int threads = sc.nextInt();
    System.out.print("Enter increments per thread: ");
    int increments = sc.nextInt();
    if (threads <= 0 || increments < 0) {
        System.out.println(
            "Invalid input. Threads must be greater than 0 "
            + "and increments cannot be negative."
        );
        sc.close();
        return;
    }
    counter.set(0);
    Thread[] threadList = new Thread[threads];
    for (int i = 0; i < threads; i++) {
        threadList[i] = new Thread
new CounterTask(increments)
    );

```

```

        threadList[i].start();
    }
    for (Thread thread : threadList) {
        thread.join();
    }
    int expected = threads * increments;
    int actual = counter.get();
    System.out.println("Expected final counter: " + expected);
    System.out.println("Actual final counter:  " + actual);
    if (expected == actual) {
        System.out.println(
            "Result: Correct - atomic operation prevented the race condition."
        );
    } else {
        System.out.println("Result: Incorrect");
    }
    sc.close();}}

```

✧ TEST CASES

S.no	Input	Test case	Expected Output
1	Threads=5 Increments = 100	Typical Case	Excepted:5000,Actual:5000
2	Threads=1 ,Increments = 1	Edge Case	Expected: 1; Actual: 1;
3	Threads=0, Increments = 100	Invalid/Boundary	Invalid input message

Test Case 1 — Typical

input

Enter number of threads: 5

Enter increments per thread: 1000

Output:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
Enter number of threads: 5
Enter increments per thread: 1000
Expected final counter: 5000
Actual final counter: 5000
Result: Correct - atomic operation prevented the race condition.

C:\Users\JAYASUDHA\OneDrive\Desktop\Java project\Main.java>
```

Test Case 2 — Edge

Input

Enter number of threads: 1

Enter increments per thread: 1

Output:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
Enter number of threads: 1
Enter increments per thread: 1
Expected final counter: 1
Actual final counter: 1
Result: Correct - atomic operation prevented the race condition.

C:\Users\JAYASUDHA\OneDrive\Desktop\Java project\Main.java>
```

Test Case 3 — Invalid/Boundary

Input

Enter number of threads: 0

Enter increments per thread: 100

Output:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
Enter number of threads: 0
Enter increments per thread: 100
Invalid input. Threads must be greater than 0 and increments cannot be negative.

C:\Users\JAYASUDHA\OneDrive\Desktop\Java project\Main.java>
```

✧ REFLECTION

I learned how `AtomicInteger` can be used when many threads work on the same counter. I understood that `incrementAndGet()` updates the counter safely and helps avoid wrong results caused by race conditions. I also learned the difference between using `AtomicInteger` and synchronized methods.

✧ DECLARATION

this assignment is my own work and has been completed as per the given requirements. The Java program, analysis, test cases, and results presented in this document are my own work. Any AI or external tool assistance used during the preparation of this assignment has been disclosed appropriately.